Media Processing Workflow Documentation

Nandini Bhimineni

Overview

The **Media Processing Workflow** in **Raritone** is responsible for handling product and user image uploads efficiently, securely, and at scale.

The workflow manages:

- Image upload handling
- Image compression
- Image resizing
- Optimized CDN delivery
- Cloud storage management
- Media cleanup handling

This architecture is designed to support AI-powered virtual try-on features while maintaining high performance, scalability, and optimized media delivery.

Technologies Used

- Node.js
- Express.js
- Multer
- Sharp
- Cloudinary CDN
- MongoDB

Purpose of the Media Processing Workflow

Large image files can:

- Slow down APIs
- Increase server load
- Consume excessive storage
- Reduce frontend performance

To solve these issues, the workflow:

- Compresses uploaded images
- Resizes images dynamically
- Optimizes media delivery
- Uses CDN-based cloud storage infrastructure

This improves:

- Upload speed
- API performance
- Frontend loading speed
- Scalability
- User experience

Workflow Architecture

User Uploads Image

↓

Multer Receives File

↓

Backend Validates File

↓

Sharp Compresses Image

↓

Image Uploaded to Cloudinary

↓

Cloudinary Generates CDN URL

↓

Optimized URL Stored in Database

↓

Frontend Receives Optimized Image

Step-by-Step Workflow

Step 1 — Image Upload

Users upload product or profile images through frontend forms.

The backend receives uploaded files using **Multer middleware**.

Example

req.files

Multer temporarily processes uploaded files in memory before further processing.

Step 2 — File Validation

The backend validates:

- File existence
- File type
- Upload structure

Example Validation

if (req.files && req.files.length > 0)

This prevents invalid or empty upload requests.

Step 3 — Image Compression

Uploaded images are compressed and resized using **Sharp**.

Example

```
const compressedBuffer =
await sharp(file.buffer)
  .resize(800)
  .jpeg({ quality: 80 })
  .toBuffer();
```

Compression Process

The workflow:

- Reduces image dimensions
- Lowers file size
- Maintains acceptable visual quality

Benefits

- Faster uploads

- Lower bandwidth usage
- Reduced storage costs
- Improved API performance

Step 4 — Cloudinary Upload

Compressed images are uploaded to the **Cloudinary CDN**.

Example

```
const result =
await uploadToCloudinary(
  compressedBuffer,
  {
    folder: "raritone/products",
    resource_type: "auto",
  }
);
```

Cloudinary Features

Cloudinary provides:

- Secure cloud storage
- CDN-based media delivery
- Dynamic image transformation
- Scalable media hosting
- Optimized global distribution

Step 5 — CDN Optimization

Optimized image URLs are generated automatically using Cloudinary transformations.

Example

```
const optimizedUrl =
result.secure_url.replace(
  "/upload/",
  "/upload/w_500,h_500,c_fill,f_auto,q_auto/"
```

);

Optimization Parameters

Parameter	Purpose
w_500	Resize image width
h_500	Resize image height
c_fill	Crop/fill image
f_auto	Automatic modern image format
q_auto	Automatic quality optimization

These transformations improve loading speed while maintaining image quality.

Step 6 — Database Storage

Optimized image URLs are stored in **MongoDB**.

Stored Data Includes

- Optimized image URLs
- Cloudinary public IDs
- Product references
- Media metadata

Benefits

This allows:

- Fast image retrieval
- Easy image updates
- Efficient deletion management
- Better storage organization

Step 7 — Cleanup Handling

If an upload operation fails:

- Uploaded Cloudinary images are removed
- Invalid media data is cleaned automatically

Example

await deleteFromCloudinary(publicId);

Benefits

This prevents:

- Unused cloud storage
- Orphan media files
- Storage waste
- Database inconsistency

Product Update Workflow

During product updates:

1. Existing images are deleted
2. New images are compressed
3. Optimized images are uploaded
4. Database records are updated

Benefits

This ensures:

- Storage consistency
- Clean media management
- Updated CDN delivery
- Efficient storage usage

Delete Product Workflow

When a product is deleted:

- Related Cloudinary images are removed
- Database records are deleted

This maintains clean cloud storage and prevents unused media accumulation.

Performance Optimization Features

1. Image Compression

Reduces image file sizes for better performance.

2. CDN Delivery

Cloudinary CDN improves global image delivery speed.

3. Automatic Format Optimization

Modern image formats are selected automatically based on browser support.

4. Lean Database Queries

Optimized queries improve product retrieval performance.

5. Reduced API Payload Size

Optimized images reduce API response sizes and bandwidth usage.

Security Considerations

The media workflow includes:

- Controlled upload handling
- Secure cloud storage
- Protected environment variables
- Cleanup handling
- Controlled API access

Future Security Enhancements

Potential future improvements include:

- MIME type validation
- Upload size restrictions
- Malware scanning
- Signed upload URLs
- Image moderation systems

Advantages of the Media Workflow

Faster Frontend Loading

Optimized images load quickly across devices and networks.

Reduced Storage Usage

Compressed images consume less cloud storage.

Better Scalability

Supports large-scale image handling efficiently.

Improved User Experience

Provides faster uploads and optimized image delivery.

AI Integration Ready

Prepared for future AI-based image processing workflows and virtual try-on systems.

Current Implementation Status

Completed Features

- Multer image handling
- Sharp image compression
- Cloudinary integration
- CDN optimization
- Optimized image delivery
- Cleanup handling
- Image update workflow
- Image deletion workflow

Successfully Tested

- Image uploads
- Optimized URL generation
- Compressed image delivery
- Cloudinary storage integration

Conclusion

The Media Processing Workflow in **Raritone** provides an optimized, scalable, and secure system for efficient image handling.

By integrating **Sharp** for image compression and **Cloudinary CDN** for optimized delivery, the workflow improves application performance, reduces storage overhead, and prepares the backend infrastructure for future AI-powered virtual try-on features and advanced media processing capabilities.